

D1_Introducció_al_Processament_d'Imatge

February 9, 2026

0.1 Processament tradicional

0.1.1 Representació d'una imatge

Una imatge digital és simplement una matriu de números on cada valor representa la intensitat d'un píxel.

* Escala de grisos: 0 (negre) - 255 (blanc)

* Color (RGB): 3 matrius (una per cada canal R, G, B)

Algunes consideracions Un gràfic de línies pot ser molt més fàcil de llegir quan afegim elements visuals com a marcadors (markers) en els punts de dades i una graella (grid) que ajudi a seguir visualment les línies respecte als valors numèrics.

No t'ho creus? Doncs deixa-m'ho mostrar-t'ho!

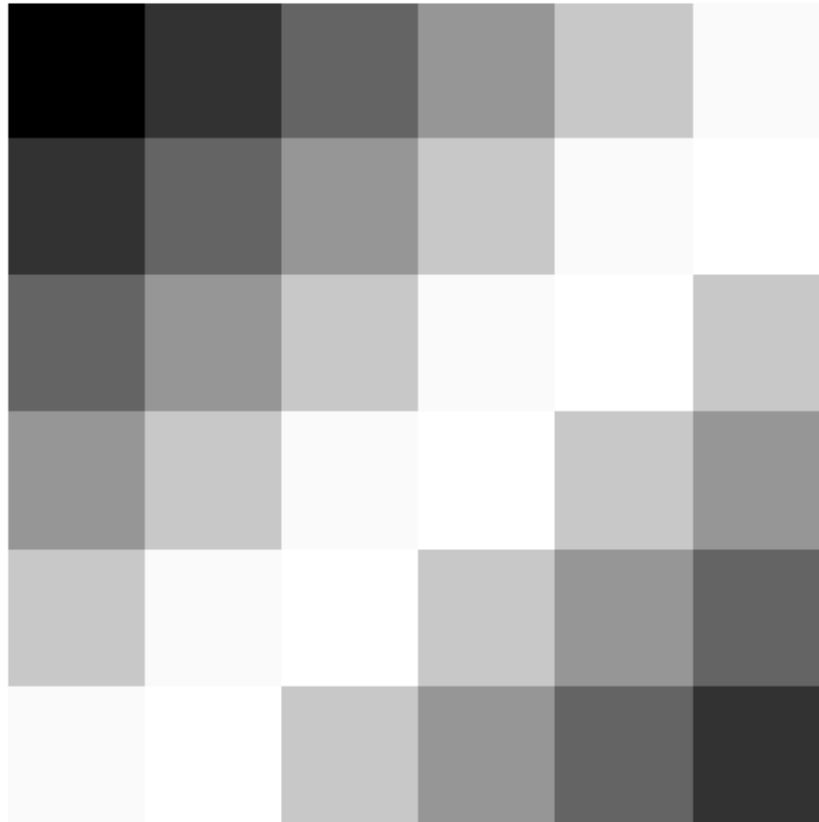
```
[1]: import numpy as np
import matplotlib.pyplot as plt

img_petita = np.array([
    [ 0,  50, 100, 150, 200, 250],
    [ 50, 100, 150, 200, 250, 255],
    [100, 150, 200, 250, 255, 200],
    [150, 200, 250, 255, 200, 150],
    [200, 250, 255, 200, 150, 100],
    [250, 255, 200, 150, 100,  50]
], dtype=np.uint8)
```

Visualitzem la imatge:

```
[2]: plt.imshow(img_petita, cmap='gray', vmin=0, vmax=255)
plt.title('Imatge 6×6 píxels', fontweight='bold', fontsize=14)
plt.axis('off')
plt.tight_layout()
plt.show()
```

Imatge 6×6 píxels



Inclús la podem guardar:

```
[3]: plt.savefig('00_imatge.png', dpi=150, bbox_inches='tight')
```

<Figure size 640x480 with 0 Axes>

Amb un sola matriu només podem crear imatges en blanc i negre. Les imatges en color son la superposició de 3 matrius: red, green, blue.

```
[4]: img_rgb = np.zeros((6, 6, 3), dtype=np.uint8)
```

```
# Fila 0: Vermell
img_rgb[0, :] = [255, 0, 0]
# Fila 1: Groc (vermell + verd)
img_rgb[1, :] = [255, 255, 0]
# Fila 2: Verd
img_rgb[2, :] = [0, 255, 0]
# Fila 3: Cian (verd + blau)
img_rgb[3, :] = [0, 255, 255]
# Fila 4: Blau
```

```

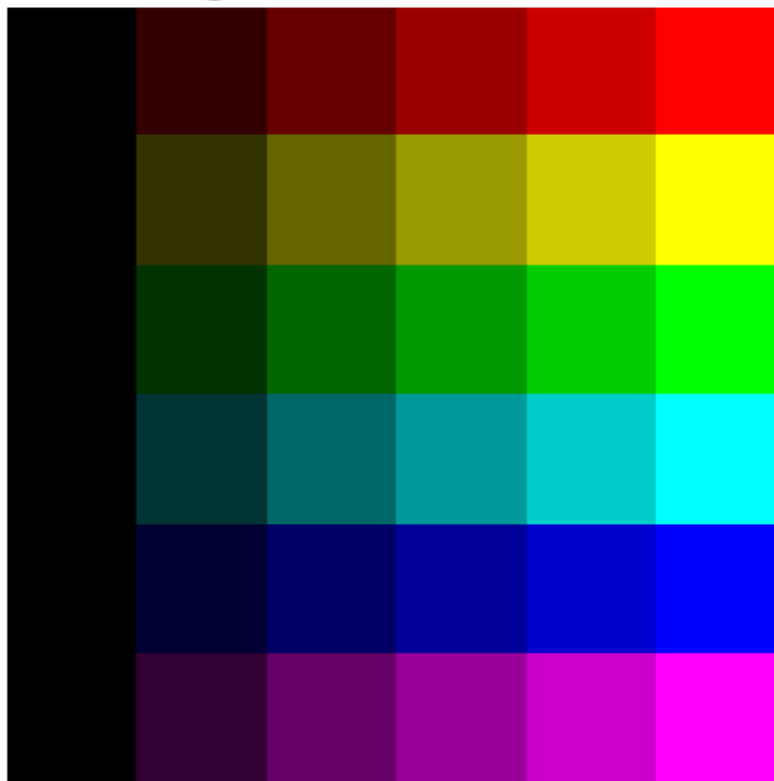
img_rgb[4, :] = [0, 0, 255]
# Fila 5: Magenta (vermell + blau)
img_rgb[5, :] = [255, 0, 255]

# Afegir gradient de brillantor
for i in range(6):
    for j in range(6):
        factor = j / 5.0 # 0.0 a 1.0
        img_rgb[i, j] = (img_rgb[i, j] * factor).astype(np.uint8)

plt.imshow(img_rgb)
plt.title('Imatge RGB 6x6 píxels\n(gradiant de colors)', fontweight='bold',
        ↪fontsize=14)
plt.axis('off')
plt.tight_layout()

```

**Imatge RGB 6x6 píxels
(gradient de colors)**



```

[5]: #Visualització dels 3 canals
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

```

```

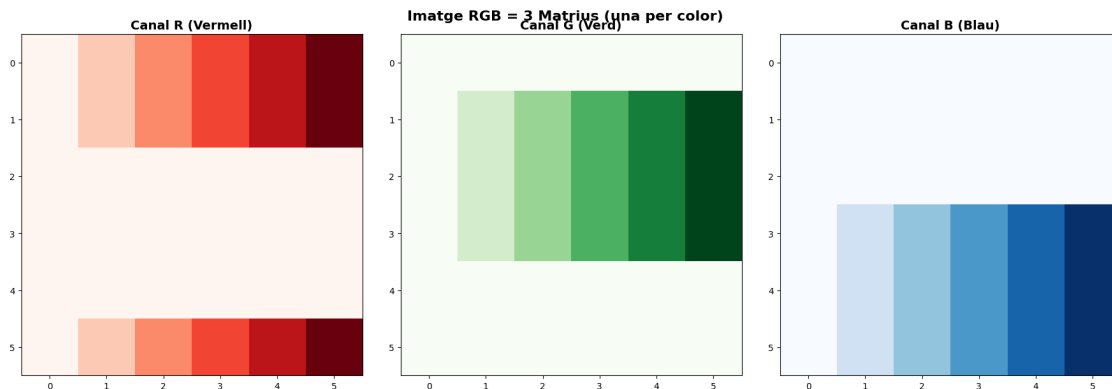
# Canal R (vermell)
axes[0].imshow(img_rgb[:, :, 0], cmap='Reds', vmin=0, vmax=255)
axes[0].set_title('Canal R (Vermell)', fontweight='bold', fontsize=14)

# Canal G (verd)
axes[1].imshow(img_rgb[:, :, 1], cmap='Greens', vmin=0, vmax=255)
axes[1].set_title('Canal G (Verd)', fontweight='bold', fontsize=14)

# Canal B (blau)
axes[2].imshow(img_rgb[:, :, 2], cmap='Blues', vmin=0, vmax=255)
axes[2].set_title('Canal B (Blau)', fontweight='bold', fontsize=14)

plt.suptitle('Imatge RGB = 3 Matrius (una per color)', fontsize=16,
             fontweight='bold')
plt.tight_layout()
plt.savefig('00m_rgb_canals.png', dpi=150, bbox_inches='tight')
plt.show()

```



Processament d'imatge tradicional Kernel de convolució

Un KERNEL (o filtre) és una petita matriu de números que aplicada a una imatge en modifica els valors (i per tant, la imatge resultant).

```

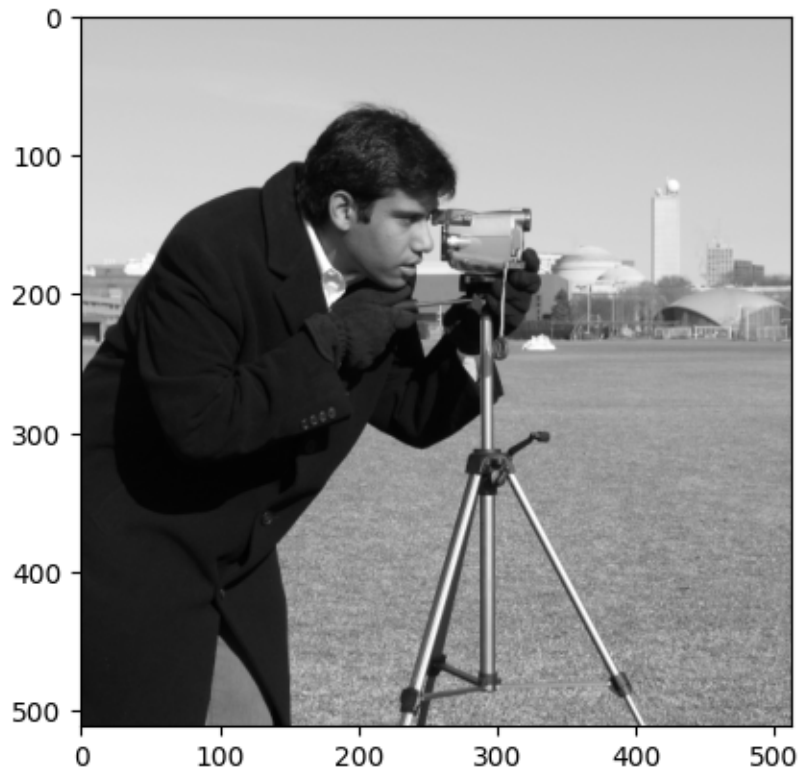
[6]: import skimage.data as skimage_data

# Carregar imatge i convertir a escala de grisos
img_camera = skimage_data.camera() # Càmera clàssica 512x512

# Reduir i fer crop per tenir una regió petita
img_conv_demo = img_camera.astype(float)
plt.imshow(img_conv_demo, cmap='gray')

```

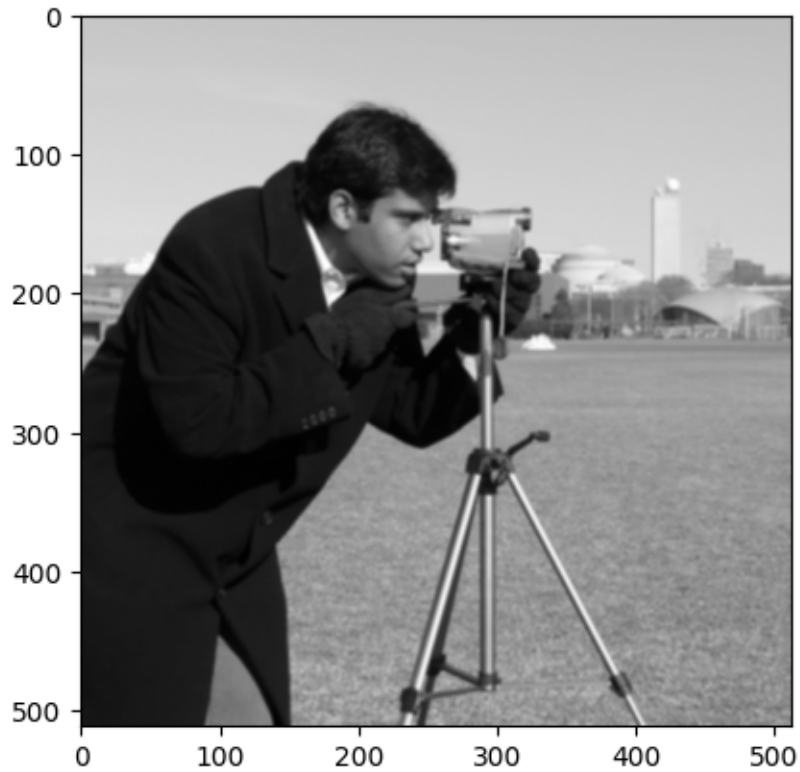
[6]: <matplotlib.image.AxesImage at 0x7d9678ebbf50>



```
[7]: # Kernel de blur
kernel_demo = np.array([
    [1, 1, 1],
    [1, 1, 1],
    [1, 1, 1]
]) / 9.0 # Blur (mitjana)
```

```
[8]: import cv2
img_amb_blur = cv2.filter2D(img_camera, -1, kernel_demo)
plt.imshow(img_amb_blur, cmap='gray')
```

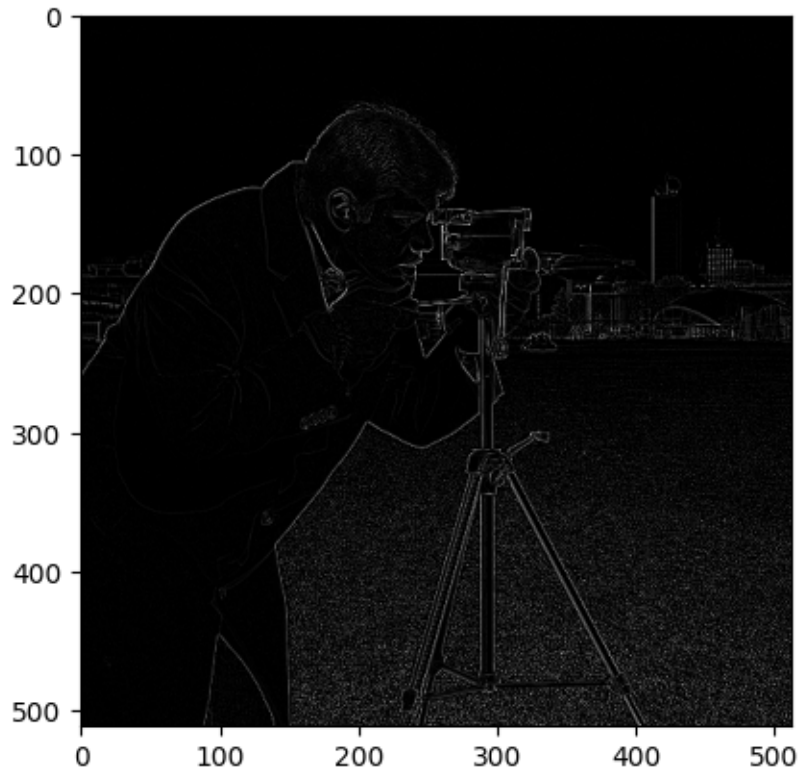
[8]: <matplotlib.image.AxesImage at 0x7d9678d07620>



```
[9]: # Kernel Laplacian (detecta vores en totes direccions)
kernel_laplacian = np.array([
    [ 0,  1,  0],
    [ 1, -4,  1],
    [ 0,  1,  0]
])

img_amb_edges = cv2.filter2D(img_camera, -1, kernel_laplacian)
plt.imshow(img_amb_edges, cmap='gray')
```

```
[9]: <matplotlib.image.AxesImage at 0x7d9678d2f620>
```

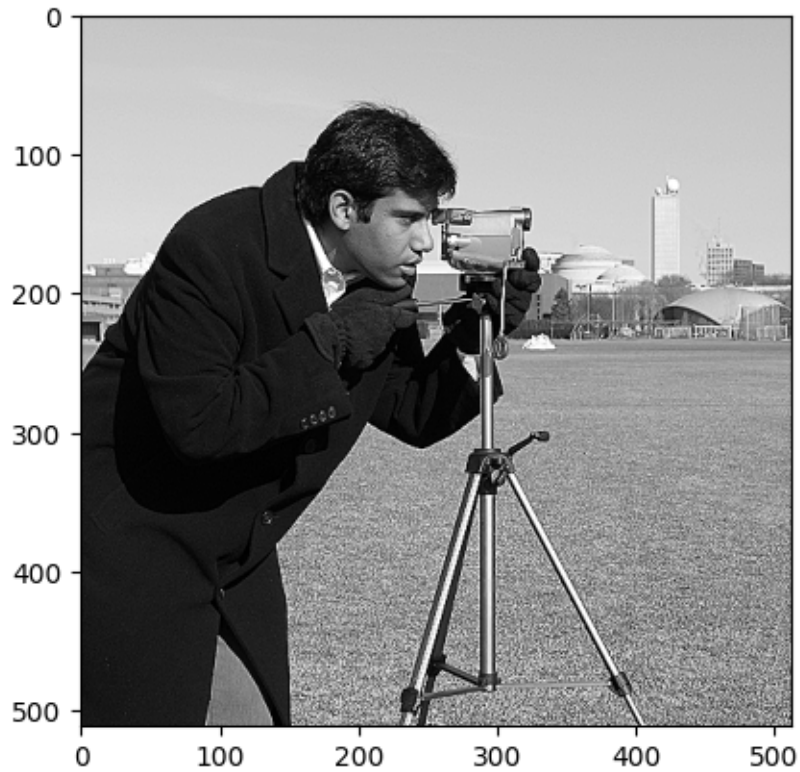


Exercici: consulta la pàgina de wikipedia ([https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing))) i prova diferents kernels i els seu efecte sobre la imatge.

```
[17]: kernel_sharpen = np.array([
    [ 0, -1, 0 ],
    [-1, 5, -1 ],
    [ 0, -1, 0 ]
])

img_amb_edges = cv2.filter2D(img_camera, -1, kernel_sharpen)
plt.imshow(img_amb_edges, cmap='gray')
```

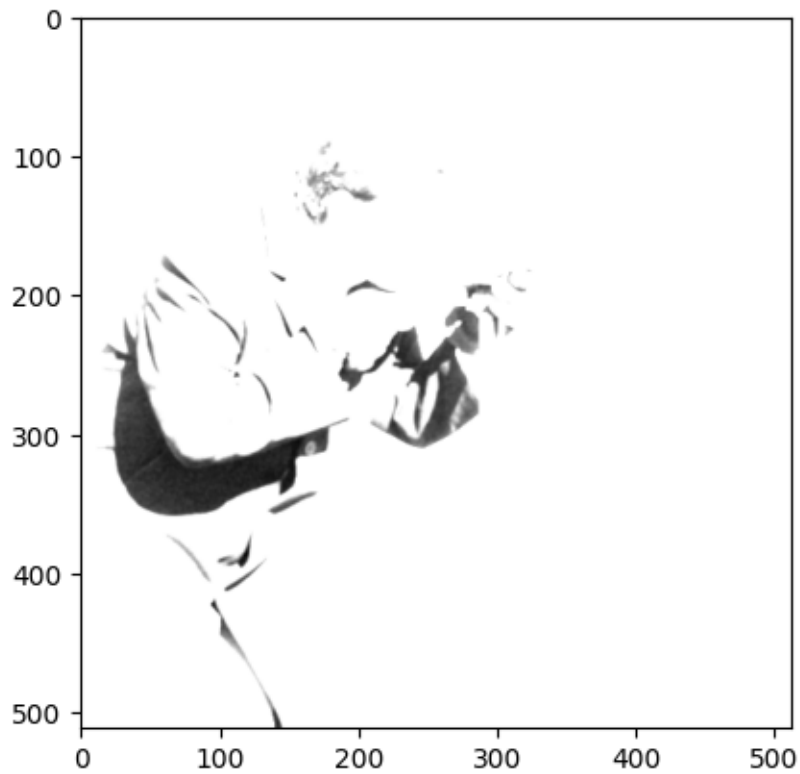
```
[17]: <matplotlib.image.AxesImage at 0x7d96789ad040>
```



```
[11]: kernel_gaussian_blur_3x3 = np.array([
        [ 1, 2, 1 ],
        [ 2, 4, 2 ],
        [ 1, 2, 1 ]
    ])

    img_amb_edges = cv2.filter2D(img_camera, -1, kernel_gaussian_blur_3x3)
    plt.imshow(img_amb_edges, cmap='gray')
```

```
[11]: <matplotlib.image.AxesImage at 0x7d9678d42600>
```

```
[16]: kernel_gaussian_blur_5x5 = np.array([
    [ 1, 4, 6, 4, 1 ],
    [ 4, 16, 24, 16, 4],
    [ 6, 24, 36, 24, 6 ],
    [ 4, 16, 24, 16, 4 ],
    [ 1, 4, 6, 4, 1 ]
])

img_amb_edges = cv2.filter2D(img_camera, -1, kernel_gaussian_blur_5x5)
plt.imshow(img_amb_edges, cmap='gray')
```

```
[16]: <matplotlib.image.AxesImage at 0x7d96789a6c90>
```

